

Практические задачи по Bash

Уровень 1: Основы

Задача 1: Информация о системе

Напишите скрипт `system_info.sh`, который выводит: - Текущую дату и время - Имя пользователя - Текущую рабочую директорию - Домашнюю директорию пользователя

Задача 2: Работа с файлами

Напишите скрипт `file_creator.sh`, который: 1. Создает директорию `test_dir` 2. Создает в ней 5 файлов: `file1.txt`, `file2.txt`, ..., `file5.txt` 3. Записывает в каждый файл строку "This is file N" (где N — номер файла) 4. Выводит список созданных файлов

Задача 3: Подсчёт файлов

Напишите скрипт `count_files.sh`, который подсчитывает: - Общее количество файлов в текущей директории - Количество `.txt` файлов - Количество `.sh` файлов

Уровень 2: Переменные и аргументы

Задача 4: Калькулятор

Напишите скрипт `calculator.sh`, который принимает три аргумента: 1. Первое число 2. Операцию (+, -, *, /) 3. Второе число

Скрипт должен выполнить операцию и вывести результат.

Пример: `./calculator.sh 10 + 5` → 15

Задача 5: Приветствие

Напишите скрипт `greet.sh`, который: - Принимает имя в качестве аргумента - Если аргумент не передан, запрашивает имя интерактивно (используя `read`) - Выводит приветствие с текущим временем суток (утро/день/вечер)

Задача 6: Работа со строками

Напишите скрипт `string_ops.sh`, который принимает строку и выводит: - Длину строки - Строку в верхнем регистре - Строку в нижнем регистре - Первые 5 символов

Уровень 3: Условия

Задача 7: Проверка файла

Напишите скрипт `check_file.sh`, который: - Принимает имя файла в качестве аргумента - Проверяет, существует ли файл - Если существует, выводит: тип (файл/директория), размер, права доступа - Если не существует, выводит сообщение об ошибке

Задача 8: Проверка числа

Напишите скрипт `check_number.sh`, который: - Принимает число в качестве аргумента - Проверяет, является ли оно положительным, отрицательным или нулём - Проверяет, чётное оно или нечётное - Проверяет, делится ли оно на 5

Задача 9: Оценка студента

Напишите скрипт `grade.sh`, который принимает оценку (0-100) и выводит буквенную оценку: - 90-100: A (Отлично) - 80-89: B (Хорошо) - 70-79: C (Удовлетворительно) - 60-69: D (Посредственно) - 0-59: F (Неудовлетворительно)

Уровень 4: Циклы

Задача 10: Таблица умножения

Напишите скрипт `multiplication_table.sh`, который: - Принимает число N в качестве аргумента - Выводит таблицу умножения для этого числа от 1 до 10 - Формат: "N x 1 = результат"

Задача 11: Обработка файлов

Напишите скрипт `process_files.sh`, который: - Находит все .txt файлы в текущей директории - Для каждого файла выводит: - Имя файла - Количество строк - Количество слов - Размер файла

Задача 12: Переименование файлов

Напишите скрипт `rename_files.sh`, который: - Находит все файлы с расширением .tmp в текущей директории - Переименовывает их, добавляя префикс "backup_" - Выводит сообщение для каждого переименованного файла

Уровень 5: Комбинированные задачи

Задача 13: Backup скрипт

Напишите скрипт `backup.sh`, который: - Принимает директорию в качестве аргумента - Создаёт архив этой директории с именем `backup_YYYY-MM-DD_HH-MM-SS.tar.gz`

- Сохраняет архив в директории ~/backups/ - Выводит сообщение об успешном создании бакапа

Задача 14: Анализ лог-файла

Создайте тестовый лог-файл `app.log` со следующим содержимым:

```
2024-01-15 10:23:45 INFO Application started
2024-01-15 10:24:12 ERROR Failed to connect to database
2024-01-15 10:24:15 WARNING Retrying connection
2024-01-15 10:24:20 INFO Connected to database
2024-01-15 10:25:30 ERROR File not found: config.xml
2024-01-15 10:26:00 INFO Processing data
2024-01-15 10:27:15 WARNING Low memory
2024-01-15 10:28:00 ERROR Out of memory
2024-01-15 10:28:30 INFO Application stopped
```

Напишите скрипт `analyze_log.sh`, который: - Подсчитывает количество записей каждого уровня (INFO, WARNING, ERROR) - Выводит все строки с ERROR - Находит первую и последнюю запись в логе - Выводит статистику в читаемом формате

Задача 15: Мониторинг дискового пространства

Напишите скрипт `disk_monitor.sh`, который: - Проверяет использование диска - Если использование превышает 80%, выводит предупреждение - Показывает топ-5 самых больших директорий в домашней папке - Результат сохраняет в файл `disk_report_YYYY-MM-DD.txt`

Уровень 6: Продвинутые задачи

Задача 16: Генератор паролей

Напишите скрипт `password_gen.sh`, который: - Принимает длину пароля в качестве аргумента (по умолчанию 12) - Генерирует случайный пароль из букв, цифр и спецсимволов - Опционально принимает количество паролей для генерации - Сохраняет пароли в файл `passwords.txt`

Пример: `./password_gen.sh 16 5` — генерирует 5 паролей длиной 16 символов

Задача 17: CSV обработчик

Создайте CSV файл `users.csv`:

```
name,age,city
Ivan,25,Moscow
Anna,30,Petersburg
Dmitry,22,Kazan
Elena,28,Moscow
Pavel,35,Petersburg
```

Напишите скрипт `csv_processor.sh`, который: - Выводит пользователей старше определённого возраста (аргумент) - Группирует пользователей по городам и считает количество - Находит самого старшего и самого молодого пользователя - Форматирует вывод в виде таблицы

Задача 18: Веб-скрапер статуса

Напишите скрипт `check_websites.sh`, который: - Принимает список URL из файла `websites.txt` (один URL на строку) - Проверяет доступность каждого сайта (используя `curl` или `wget`) - Выводит статус: доступен (HTTP 200) или недоступен - Замеряет время ответа - Результаты сохраняет в `status_report.txt` с временной меткой

Бонусная задача

Задача 19: Интерактивное меню

Напишите скрипт `menu.sh` с интерактивным меню:

```
=== Система управления файлами ===
1. Показать файлы в текущей директории
2. Создать новую директорию
3. Удалить файл
4. Найти файл по имени
5. Показать использование диска
6. Выход
```

Выберите опцию:

Скрипт должен: - Отображать меню в цикле - Выполнять соответствующие действия - Обработать ошибки (например, попытка удалить несуществующий файл) - Просить подтверждение для опасных операций (удаление) - Возвращаться в меню после каждой операции

Рекомендации по выполнению

1. Начните с простых задач и постепенно переходите к более сложным
2. Тестируйте скрипты с разными входными данными
3. Используйте **ShellCheck** для проверки кода
4. Добавляйте комментарии к неочевидным частям кода
5. Обработывайте ошибки — проверяйте корректность входных данных
6. Следуйте **best practices** — кавычки вокруг переменных, `set -euo pipefail`, и т.д.

Удачи!